

ENTERPRISE SECURITY ASSESSMENT LAB

IAM PERMISSIONS ENUMERATION

Professional methodology report for identity, managed policy, and inline policy review

Module	Cloud Security Assessment
Document Type	Individual Methodology Report
Phase	Phase 2
Evidence Source	iam-permissions-enumeration.png
Publication Type	GitHub Portfolio - Redacted Lab Version
Prepared For	04-cloud-security/README.md documentation set
File Name	02-iam-permissions-enumeration-methodology-report.pdf

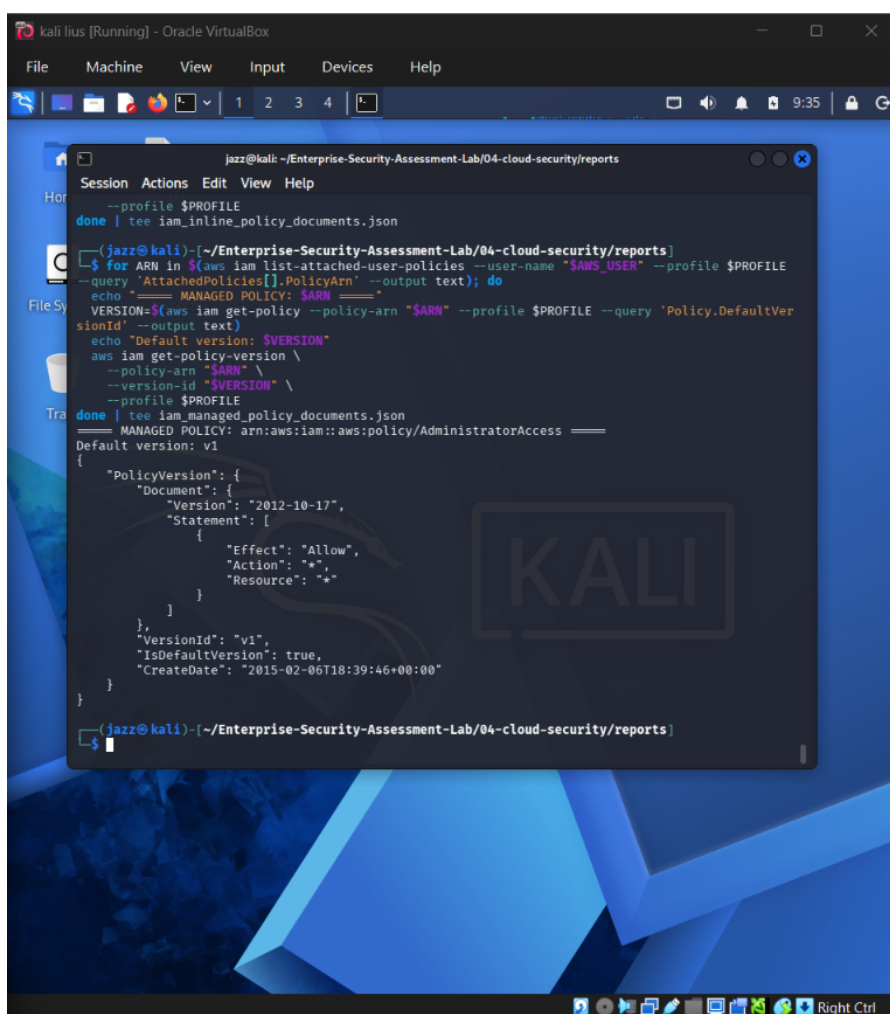
1. Executive Purpose

IAM enumeration is one of the most important parts of cloud security testing because cloud privilege escalation usually depends on permission combinations rather than a single obvious vulnerability. This report explains how the starting identity was reviewed and how policy output was interpreted defensively.

2. Scope and Evidence Mapping

Testing objective	Identify what permissions the starting CloudGoat/lab identity has and determine whether the policy set creates privilege escalation paths.
Cloud platform	Amazon Web Services (AWS)
Lab context	CloudGoat / authorized vulnerable cloud lab environment
Primary tools	AWS CLI, IAM API, jq / terminal review
Evidence file(s)	iam-permissions-enumeration.png
Result type	IAM identity and policy review completed

3. Embedded Evidence Screenshot(s)

The screenshot shows a Kali Linux terminal window titled 'jazz@kali: ~/Enterprise-Security-Assessment-Lab/04-cloud-security/reports'. The terminal displays a series of commands and their outputs for enumerating IAM permissions. The commands include: 1. A loop to list attached user policies for 'SAWS_USER' and save them to 'iam_inline_policy_documents.json'. 2. A loop to get the policy version for each policy and save the results to 'iam_managed_policy_documents.json'. 3. A final command to get the policy document for the 'AdministratorAccess' policy. The output shows the policy version 'v1' and the policy document, which includes a 'Statement' with an 'Effect' of 'Allow' and an 'Action' of '*' on the resource '*'. A large 'KALI' watermark is visible in the background of the terminal window.

```
jazz@kali: ~/Enterprise-Security-Assessment-Lab/04-cloud-security/reports
--profile $PROFILE
done | tee iam_inline_policy_documents.json

(jazz@kali)~/Enterprise-Security-Assessment-Lab/04-cloud-security/reports
$ for ARN in $(aws iam list-attached-user-policies --user-name "SAWS_USER" --profile $PROFILE
--query 'AttachedPolicies[].PolicyArn' --output text); do
echo "===== MANAGED POLICY: $ARN ====="
VERSION=$(aws iam get-policy --policy-arn "$ARN" --profile $PROFILE --query 'Policy.DefaultVer
sionId' --output text)
echo "Default version: $VERSION"
aws iam get-policy-version \
--policy-arn "$ARN" \
--version-id "$VERSION" \
--profile $PROFILE
done | tee iam_managed_policy_documents.json
===== MANAGED POLICY: arn:aws:iam::aws:policy/AdministratorAccess =====
Default version: v1
{
  "PolicyVersion": {
    "Document": {
      "Version": "2012-10-17",
      "Statement": [
        {
          "Effect": "Allow",
          "Action": "*",
          "Resource": "*"
        }
      ]
    },
    "VersionId": "v1",
    "IsDefaultVersion": true,
    "CreateDate": "2015-02-06T18:39:46+00:00"
  }
}
```

Figure 1 - IAM policy document review showing administrator-style permissions in the lab output.

4. Methodology and Execution Workflow

The tester confirms the current identity, identifies the IAM user where applicable, lists attached managed policies, lists inline policy names, and retrieves inline policy documents. The output is reviewed for wildcard actions, privilege-sensitive IAM permissions, PassRole risk, EC2 launch capability, and policy attachment capability.

Command / workflow used

```
aws sts get-caller-identity --profile $PROFILE --region $REGION
aws iam get-user --profile $PROFILE
aws iam list-attached-user-policies --user-name <REDACTED_USER> --profile $PROFILE
aws iam list-user-policies --user-name <REDACTED_USER> --profile $PROFILE
aws iam get-user-policy --user-name <REDACTED_USER> --policy-name <REDACTED_POLICY_NAME> --profile $PROFILE
```

Execution notes

- Review both managed policies and inline policies.
- Look for wildcard Action and Resource combinations.
- Identify privilege-sensitive actions such as iam:PassRole and policy modification actions.
- Record the evidence as a policy review, not as a production finding.

5. Sanitized Tool Output Style

The following output example is sanitized. It documents the expected evidence style without exposing AWS account IDs, ARN values, AccessKeyId values, SecretAccessKey values, session tokens, private bucket names, or billing details.

```
Managed policy review:
- PolicyArn: arn:aws:iam::aws:policy/<POLICY_NAME>
- Effect: Allow
- Action: *
- Resource: *
```

Interpretation: lab identity has highly permissive policy material suitable for CloudGoat testing.

6. Professional Interpretation

The evidence shows IAM permissions being enumerated and policy material being reviewed. In a vulnerable lab this is expected, but in a real account wildcard privileges or administrator-level policy attachments would be high risk.

7. Risk Analysis

Risk Area	Explanation
Privilege escalation risk	Overly broad policies can allow a low-privileged principal to become administrator.
PassRole risk	iam:PassRole combined with EC2 or Lambda creation can enable role abuse.
Policy modification risk	AttachPolicy, PutPolicy, or CreatePolicyVersion permissions can change effective privilege levels.

8. Defensive Recommendations

- Apply least privilege to all IAM identities.
- Avoid Action:* and Resource:* except for tightly controlled admin roles.
- Restrict iam:PassRole to specific approved roles.
- Use permission boundaries for lab users and service roles.
- Monitor IAM policy changes using CloudTrail, Config, GuardDuty, or Security Hub.

9. GitHub Redaction and Publication Guidance

Before publishing this document or the underlying screenshot evidence, verify that the following values are removed, blurred, or replaced with placeholders. This is especially important because cloud evidence can contain values that are valid beyond the lab session if not rotated or destroyed.

Sensitive value	Required handling
AWS Account ID	Blur or replace with <REDACTED_ACCOUNT_ID>.
ARN values	Redact account number and private role/user names where needed.
AccessKeyId / SecretAccessKey	Must never be visible. Rotate immediately if exposed.
SessionToken	Must be fully redacted because temporary credentials can still be abused during validity period.
Bucket names / resource names	Redact if they identify private infrastructure or are still active.

10. Evidence Quality Checklist

- The screenshot is readable and aligned within the page margins.
- The evidence filename matches the GitHub link exactly.
- The command/output supports the stated result type.
- No AWS credentials, session tokens, account numbers, or private resource identifiers are visible.
- The report explains both the offensive testing value and the defensive remediation value.